

Day 6 - Facilitation Guide

(Using CSS and Javascript)

INDEX

- I. Recap of the previous session**
- II. External CSS**
- III. Introduction to Bootstrap**
- IV. Colors and Backgrounds**
- V. Scripting languages**
- VI. Introduction to JavaScript**
- VII. data types**
- VIII. Variables**
- IX. Operators and expressions**
- X. HTML event attributes**

(2 hrs) ILT

I. Recap of the previous session

In our previous session, we had a detailed discussion on how to use multimedia and CSS in HTML5 documents. HTML5 provides many useful tags to include multimedia items in an HTML document. HTML provides the following tags to embed multimedia content directly into the Webpages.

- : Used for embedding images.
- <audio controls> , <audio>: Used for embedding audio files.
- <video>: Used for embedding video files.
- <canvas>: Used for drawing graphics and animations using JavaScript.
- <svg> : Used for creating scalable vector graphics.

Please consider browser compatibility when working with multimedia in HTML5.

In addition to that we have also explained in detail about CSS (Cascading Style Sheets) and how to use it with HTML. CSS provides user capabilities to add style to our Webpages to make them more attractive and user friendly.

In this session we will discuss some other ways of using CSS. We will also introduce JavaScript and explain how to use JavaScript inside an html page.

II. External CSS

The external style sheet is generally used when you want to make changes on multiple pages. It is ideal for this condition because it facilitates you to change the look of the entire web site by changing just one file.

It uses the <link> tag on every page and the <link> tag should be put inside the head section.

Let's take an example of a style sheet file named "mystyle.css".

```
body {
    background-color: lightblue;
}
h1 {
    color: navy;
    margin-left: 20px;
}
```

Referencing the external style sheet in an html page within the <link> tag.

```
<head>
<!-- Using an external stylesheet called mystyle.css -->
<link rel="stylesheet" type="text/css" href="mystyle.css">
</head>
```

Let's go through an complete example:

First create an html file (index.html):

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Example</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <h1>External Styles</h1>
    <p id="intro">Allow you to define styles for the whole website.</p>
    <p class="colorful">This has a style applied via a class.</p>
  </body>
</html>
```

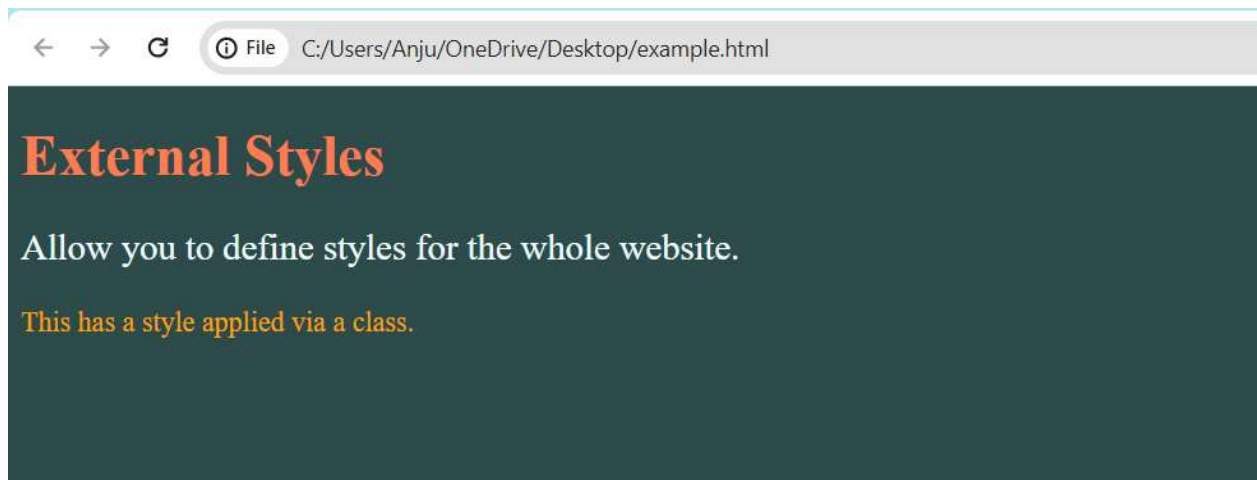
Now, create a .css file as styles.css

```
/*external style coding*/
body {
```

```
background-color: darkslategrey;
color: azure;
font-size: 1.1em;
}
h1 {
color: coral;
}
#intro {
font-size: 1.3em;
}
.colorful {
color: orange;
}
```

Here,
#intro - used to set style for id="intro"
.colorful - used to set style for class="colorful"

Output:



In this example:

The HTML file (index.html) contains the structure of the Webpage.

- The <link> tag in the <head> section links the HTML file to an external CSS file (styles.css). The href attribute specifies the path to the CSS file.
- The CSS file (styles.css) contains the styling rules for the HTML document.
- Styles in the CSS file are written using the same syntax as embedded CSS, but they are separated into their own file.
- The styles defined in styles.css apply to the HTML elements in index.html based on the selectors.

We used style for body, header and paragraphs. If we save the above files in a directory and open the HTML page then we will see.

Advantages of External CSS:

Maintainability: Styles are centralized in a separate file, making it easier to manage and update styles across multiple pages.

Reusability: The same external CSS file can be linked to multiple HTML pages, promoting code reusability.

Separation of Concerns: There is a clear separation between the HTML structure and the styling, making the code more modular and improving collaboration among developers.

Using external CSS is a best practice for larger projects or Websites with multiple pages, as it promotes a more organized and scalable code structure.

III. Introduction to Bootstrap

Bootstrap is a free, open source front end development framework for the creation of websites and web apps. It is the most popular HTML, CSS and JavaScript framework for developing a responsive and mobile friendly website. It is absolutely free to download and use.

Let's see an example to use bootstrap in an html coding:

index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <title>Bootstrap Example</title>
```

```
  <meta charset="utf-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
  <!--bootstrap link-->
```

```
  <link rel="stylesheet"
```

```
    href="https://cdn.jsdelivr.net/npm/bootstrap@4.4.1/dist/css/bootstrap.min.css"
```

```
    integrity="sha384-Vkoo8x4CGsO3+Hhvx8T/Q5PaXtkKtu6ug5TOeNV6gBiFeWPGFN9MuhOf23Q9Ifjh" crossorigin="anonymous">
```

```
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
```

```
    <script
```

```
      src="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/js/bootstrap.min.js"></script>
```

```
  </head>
```

```
<body>
```

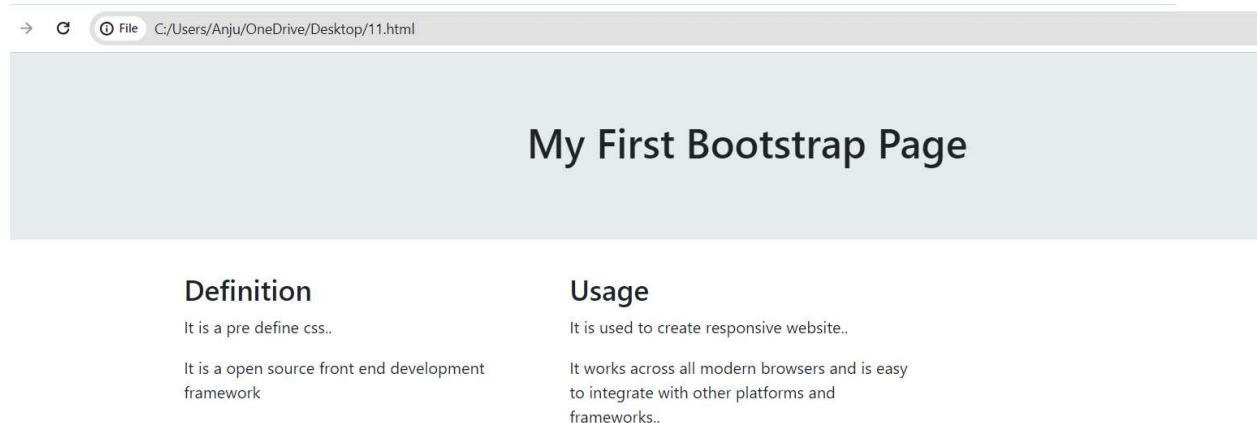
```
<div class="jumbotron text-center">
```

```

<h1>My First Bootstrap Page</h1>
</div>
<div class="container">
  <div class="row">
    <div class="col-sm-4">
      <h3>Definition</h3>
      <p>It is a predefined css..</p>
      <p>It is a open source front end development framework</p>
    </div>
    <div class="col-sm-4">
      <h3>Usage</h3>
      <p>It is used to create responsive website..</p>
      <P>It works across all modern browsers and is easy to integrate with other
platforms and frameworks..</P>
    </div>
  </div>
</div>
</body>
</html>

```

Output:



IV. Colors and Backgrounds

HTML provides us with mechanisms to change the color of the HTML widgets and to set the background color of a Web page or an element on the Web page. Setting background color and widgets color enable us to create unique and attractive layouts for the Web page. HTML also allows us to use a background image which will make the page look more appealing.

To set the background color in HTML, we use the style attribute, with the CSS property background-color inside the body tag of the HTML document. We can change the background color by overriding the default property with the other property.

Note: HTML5 does not support the “bgcolor” attribute inside the <body> tag, so the CSS style is used to add background color. The “bgcolor” attribute is deprecated in HTML5 and should not be used even though it is supported in some browsers.

In CSS, color values can be expressed in several formats, allowing you to specify colors in different ways. Here are the common ways to use color values in CSS:

1. Color Keywords:

You can use predefined color names in CSS. For example:

```
body {  
    color: red;  
    background-color: lightblue;  
}
```

Common color keywords include red, blue, green, yellow, black, white, etc.

2. Hexadecimal Notation:

Hexadecimal notation is one of the most widely used formats for specifying colors in CSS. It uses a six-digit code representing the intensity of red, green, and blue (RGB) in hexadecimal. For example:

```
h1 {  
    color: #ff0000; /* Red */  
    background-color: #00ff00; /* Green */  
}
```

Each pair of digits represents one of the RGB components (red, green, blue), and valid values range from 00 to FF. You can also use color picker in the editor which will convert the color to hex value.

3. RGB Function:

The rgb() function allows you to specify a color using the intensity of red, green, and blue as decimal values. For example:

```
p {  
    color: rgb(255, 0, 255); /* Magenta */  
    background-color: rgb(128, 128, 0); /* Olive */  
}
```

The values for red, green, and blue can range from 0 to 255.

4. RGBA Function:

Similar to `rgb()`, the `rgba()` function allows you to specify color with an additional alpha channel for transparency. The alpha channel ranges from 0 (fully transparent) to 1 (fully opaque). For example:

```
div {  
    background-color: rgba(0, 128, 255, 0.7); /* Semi-transparent blue */  
}
```

5. HSL Notation:

The `hsl()` function allows you to specify a color using the hue, saturation, and lightness. Hue is represented as an angle in degrees (0 to 360), saturation and lightness as percentages (0% to 100%). For example:

```
a {  
    color: hsl(120, 100%, 50%); /* Green */  
    background-color: hsl(0, 100%, 50%); /* Red */  
}
```

6. HSLA Notation:

Similar to `hsl()`, the `hsla()` function includes an alpha channel for transparency. For example:

```
span {  
    background-color: hsla(240, 100%, 50%, 0.5); /* Semi-transparent blue */  
}
```

CSS color schemes

When we develop a Webpage we should carefully choose a color scheme to contribute to brand identity, improve readability, and enhance the overall user experience.

CSS color schemes refer to predefined sets of colors that are used consistently throughout a Website or Web application.

These color schemes help maintain a harmonious and visually appealing design by establishing a cohesive and consistent color palette. A well-defined color scheme is a vital element of good Web design.

Common types of CSS color schemes include:

Monochromatic:

A monochromatic color scheme uses variations in lightness and saturation of a single color. It creates a harmonious and unified look.



Here's an HTML page using shades of blue:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Monochromatic Color Scheme Example</title>
  <!-- Linking to External CSS File -->
  <link rel="stylesheet" type="text/css" href="styles.css">
</head>
<body>
  <header>
    <h1>Monochromatic Color Scheme Example</h1>
  </header>
  <section>
    <p>This is a paragraph of text with a monochromatic color scheme.</p>
  </section>
</body>
</html>
```

Here is css file :

```
/* External CSS File */

/* Monochromatic Color Scheme */
header {
  background-color: #3498db; /* Blue */
  color: #fff; /* White text on blue background */
}
```



```

}

section {
  background-color: #5dade2; /* Lighter Blue */
  color: #333; /* Dark Gray text on lighter blue background */
}

p {
  border-left: 4px solid #2e86c1; /* Accent color for paragraph border */
  padding-left: 10px;
}

```

If you open the index.html in browser you will see a page like this:



You can see that in this monochromatic color scheme, different shades, tones, and tints of a single color are used.

Analogous:

Analogous color schemes use colors that are next to each other on the color wheel. They provide a sense of unity and are often found in nature.



Below is an example of HTML and CSS code that represents an analogous color scheme. In this example, I'll use shades of blue to create an analogous color scheme.

HTML File (index.html):

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Example</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
<header>
  <h1>Analogous Color Scheme Example</h1>
  <header>
    <h2>Orange shade</h2>
  </header>
  <section>
    <p>Yellow shade</p>
  </section>
</body>
</html>
```

CSS File (styles.css):

```
/* External CSS File */

/* Analogous Color Scheme */
header {
  background-color:#FFAA33; /* Orange */
  color: #fff; /* White text on blue background */
}

section {
  background-color: #FAFA33; /* yellow*/
  color: #fff; /* White text on reddish orange background */
}

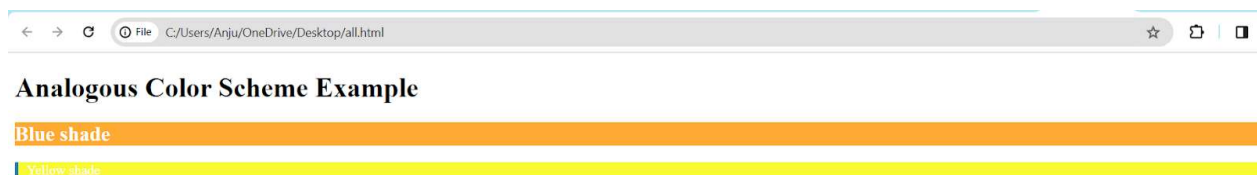
p {
  border-left: 4px solid #2e86c1; /* Accent color for paragraph border */
  padding-left: 10px;
}
```

In this example:

- The HTML file (index.html) links to an external CSS file (styles.css).

- The external CSS file contains styles for the <header>, <section>, and <p> elements, applying an analogous color scheme using shades of blue.
- The <header> has a blue background with white text.
- The <section> has a lighter blue background with dark gray text.
- The <p> element has a left border with an accent color to add visual interest.

If you open the index.html in browser you will see a page like this:



You can adjust the color values to explore different shades within the analogous color scheme or modify the HTML structure and CSS styles based on your design preferences.

Complementary:

Complementary colors are opposite each other on the color wheel. They create a high contrast and vibrant look when used together.



In a complementary color scheme, two colors that are opposite each other on the color wheel are used. Here's an example of HTML page using blue and orange:

HTML File (index.html):

```
<!DOCTYPE html>
<html>
  <head>
```

```

        <title>My Example</title>
        <link rel="stylesheet" href="style.css">
    </head>
    <body>
<header>
    <h1>Complementary Color Scheme Example</h1>
<header>
    <h2>Purple shade</h2>
</header>
<section>
    <p>Orange shade</p>
</section>

</body>
</html>

```

CSS File (styles.css):

```

/* External CSS File */

/* Complementary Color Scheme */
header {
    background-color:#800080; /* Purple*/
    color: #fff; /* White text on blue background */
}

section {
    background-color: #FFAA33; /* Orange */
    color: #fff; /* White text on reddish orange background */
}

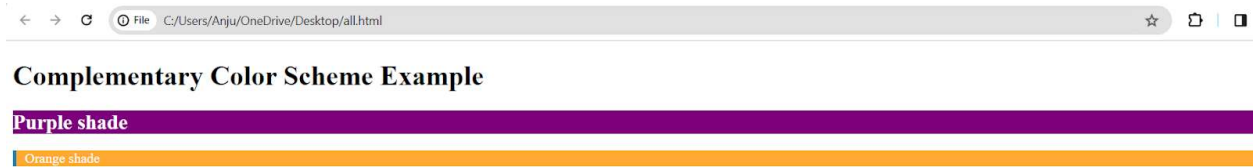
p {
    border-left: 4px solid #2980b9; /* Accent color for paragraph border (darker
blue) */
    padding-left: 10px;
}

```

In this example:

- The HTML file (index.html) links to an external CSS file (styles.css).
- The external CSS file contains styles for the <header>, <section>, <p>, and <button> elements, applying a complementary color scheme using blue and reddish orange.
- The <header> has a blue background with white text.
- The <section> has a reddish-orange background with white text.
- The <p> element has a left border with an accent color (darker blue) to add visual interest.

If you open the index.html in browser you will see a page like this:



Now experiment with different complementary color pairs or modify the HTML structure and CSS styles based on your design preferences and get a different look and feel of your developed pages.

V. Scripting languages

A script or scripting language is a computer language that does not need the compilation step and is rather interpreted one by one at runtime. It is where the script is written and where instructions for a run-time environment are written. In contrast to programming languages that are compiled first before running, scripting languages do not compile the file and execute the file without being compiled.

Types of scripting languages:

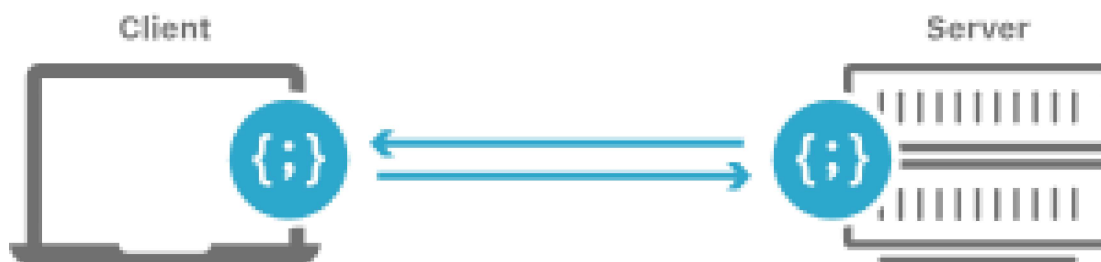
Client-side scripting

Client-side scripting simply means running scripts, such as JavaScript, on the client device, usually within a browser. All kinds of scripts can run on the client side if they are written in JavaScript, because JavaScript is universally supported.

Server-side scripting

Server-side scripts run on the server instead of the client, often in order to deliver dynamic content to web pages in response to user actions. Server-side scripts don't have to be written in JavaScript, since the server may support a variety of languages.

Scripts run client-side and server-side:



Data passes back and forth between the client and server, and scripts can run on both sides.

Note: A server is a computer or system that provides resources, data, services, or programs to other computers, known as clients, over a network.

There are multiple scripting languages available some are as follows:

- JavaScript: Primarily used for web development, it enables dynamic content on websites and runs in web browsers.
- Python: Known for its readability and versatility, used in web development, data analysis, artificial intelligence, and more.
- Ruby: Emphasizes simplicity and productivity, often used in web development.
- PHP: Specifically designed for server-side web development to create dynamic web pages.
- Bash (Shell scripting): Used for automating tasks in Unix-like operating systems.

Application of Scripting Languages:

- Scripting languages are used on the server-side as well as the client-side and are also widely used in web applications. The languages like JavaScript, Python, PHP, and Perl are examples of server-side scripting languages, and JavaScript, jQuery, AJAX are examples of client-side scripting languages.
- Like Python scripts, Perl, Shell are all the scripting languages, which are used in system administration.
- It is used to construct plugins and extensions for current applications, as well as in games and multimedia.

VI. Introduction to JavaScript

JavaScript is a versatile programming language and it is commonly used in HTML pages. It is a client side scripting language.

Here are some key reasons why JavaScript is often required in HTML pages:

Interactivity: JavaScript allows developers to create interactive and dynamic Web pages. It enables the manipulation of HTML elements, handling user input, and responding to events such as clicks, form submissions, and keyboard interactions. This interactivity enhances the user experience and makes Web pages more engaging.

Client-Side Scripting: JavaScript is a client-side scripting language, meaning it runs in the user's browser rather than on the server. This enables Web developers to create responsive and fast user interfaces without relying on server requests for every interaction.

DOM Manipulation: JavaScript provides the ability to manipulate the Document Object Model (DOM), which represents the structure of an HTML document. Developers can

dynamically change the content, structure, and style of a Webpage in response to user actions or other events.

Form Validation: JavaScript is often used to perform client-side form validation before submitting data to the server. This helps improve user experience by providing instant feedback on form input errors without the need for a server round-trip.

Asynchronous Operations: JavaScript supports asynchronous programming, allowing developers to perform tasks such as making AJAX (Asynchronous JavaScript and XML) requests. This enables the retrieval of data from a server without refreshing the entire page, leading to a smoother user experience.

Browser Compatibility: JavaScript is supported by all modern Web browsers, making it a cross-browser scripting language. This ensures that interactive features work consistently across different browsers and platforms.

Animations and Effects: JavaScript can be used to create animations and visual effects on Web pages. This includes transitions, fades, slides, and other dynamic elements that enhance the visual appeal of a Website.

Dynamic Content Loading: With JavaScript, Web pages can load additional content dynamically without requiring a full page reload. This is often used in single-page applications (SPAs) to create a seamless browsing experience.

Cookies and Local Storage: JavaScript enables the manipulation of browser cookies and local storage, allowing developers to store and retrieve data on the client side. This is useful for maintaining user preferences, session information, and other client-specific data.

Third-Party Integration: JavaScript facilitates the integration of third-party APIs (Application Programming Interfaces) and services, allowing developers to incorporate features such as maps, social media feeds, and analytics into their Web pages.

In summary, JavaScript is a crucial component of modern Web development, providing the means to create dynamic, interactive, and feature-rich Web pages that meet the expectations of today's users.

Let's see an example of client side scripting using javascript.

HTML File (index.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Client-Side JavaScript Example</title>
```

```
<!-- Linking to External JavaScript File -->
<script src="script.js" defer></script>
</head>

<body>
  <h1 id="greeting">Hello, World!</h1>

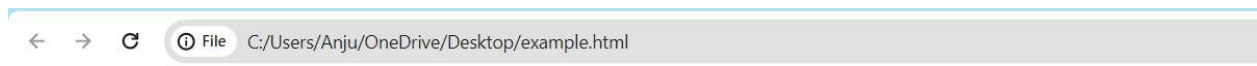
  <button onclick="changeGreeting()">Change Greeting</button>
</body>
</html>
```

JavaScript File (script.js):

```
// External JavaScript File
function changeGreeting() {
  // Get the element with the id 'greeting'
  var greetingElement = document.getElementById('greeting');

  // Check if the element exists
  if (greetingElement) {
    // Change the text content of the element
    greetingElement.textContent = 'Welcome to Client-Side JavaScript!';
  }
}
```

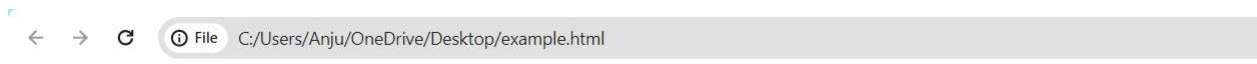
The page will look like this in the browser:



Javascript example

Change Greeting

If you click on the button “Change Greeting” the page appearance will change.



Welcome to Client-Side JavaScript!

Change Greeting

In this example:

- The HTML file (index.html) includes a <script> tag that links to an external JavaScript file (script.js).
- The HTML body contains an <h1> element with the id 'greeting' and a <button> element with an “onclick” attribute calling the changeGreeting() function.
- The JavaScript file (script.js) defines the changeGreeting() function, which retrieves the element with the id 'greeting' and changes its text content to 'Welcome to Client-Side JavaScript!' when the button is clicked.

VII. Data Types:

JavaScript provides different data types to hold different types of values.

Primitive Data Types:

String:

```
let str = 'Hello, World!';
```

Number:

```
let num = 42;
```

Boolean:

```
let isTrue = true;  
let isFalse = false;
```

Undefined:

```
let undefinedVar;
```

Null:

```
let nullVar = null;
```

Symbol (ES6 and later):

```
let symbolVar = Symbol('uniqueSymbol');
```

Composite Data Type:

Object:

```
let person = {  
  name: 'John',  
  age: 25,  
  isStudent: false  
};
```

Array:

```
let colors = ['red', 'green', 'blue'];
```

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JavaScript Basics</title>
</head>

<body>
  <script>
    // Basic Syntax
    console.log('Hello from JavaScript!');

    let x = 5;
    if (x > 0) {
      console.log('Positive');
    } else {
      console.log('Non-positive');
    }

    // Data Types
    let message = 'Hello, World!';
    let age = 30;
    let isStudent = false;

    console.log(message, age, isStudent);

    let colors = ['red', 'green', 'blue'];
    console.log(colors);

    let person = {
      name: 'Alice',
      age: 25,
      isStudent: true
    };
    console.log(person.name, person.age, person.isStudent);
  </script>
</body>
```

</html>

Output:



This HTML file includes basic JavaScript code within the `<script>` tag. It demonstrates the basic syntax and data types discussed above. Open this file in a Web browser, and you should see the output in the browser console.

VIII. Variables

In JavaScript, variables are used to store and manage data. They are declared using keywords such as `var`, `let`, or `const`. The scope of a variable determines where in the code it can be accessed or modified. JavaScript has two types of variable scope: global scope and local scope.

Variable Declaration:

var:

Historically used for variable declaration.

Has function scope.

Can be re-declared and updated.

Example:

```
var age = 25;
```

let:

Introduced in ES6 (ECMAScript 2015) for block-scoped variables.

Has block scope.

Can be updated but not re-declared.

Example:

```
let name = 'John';
```

const:

Also introduced in ES6 for block-scoped constants.

Has block scope.

Cannot be updated or re-declared after initialization.

Example:

```
const PI = 3.14;
```

Variable Scope

Global Scope:

Variables declared outside any function or block have global scope.
Accessible throughout the entire program.

Example:

```
var globalVar = 'I am global';
```

Local Scope (Function Scope):

Variables declared inside a function have local scope.
Accessible only within that function.

Example:

```
function myFunction() {  
    var localVar = 'I am local';  
    console.log(localVar);  
}
```

Block Scope (let and const):

Variables declared with let and const have block scope.
Accessible only within the block where they are defined.

Example:

```
if (true) {  
    let blockVar = 'I am in a block';  
    console.log(blockVar);  
}
```

Hoisting

JavaScript has a behavior called hoisting, where variable declarations are moved to the top of their containing scope during the compilation phase. However, only the declaration is hoisted, not the initialization.

Example of Hoisting:

```
console.log(x); // Output: undefined  
var x = 5;  
console.log(x); // Output: 5
```

In the first console.log, x is hoisted, but its value is undefined until the line var x = 5 is executed.

Best Practices: Prefer using let and const over var for better scoping.

Declare variables at the top of their scope to avoid unexpected behavior due to hoisting. Minimize the use of global variables to prevent unintended interactions.

IX. Operators and expressions

In JavaScript, operators are symbols that perform operations on operands. Operands can be values or variables. Expressions are combinations of operators and operands that produce a value. Javascript operators have the similar syntax like Java operators.

Let's explore various types of operators and expressions in JavaScript:

Arithmetic Operators:

Arithmetic operators perform basic mathematical operations.

```
let x = 10;
let y = 5;

let sum = x + y; // Addition
let difference = x - y; // Subtraction
let product = x * y; // Multiplication
let quotient = x / y; // Division
let remainder = x % y; // Modulus (remainder)
let increment = x++; // Increment (post-increment)
let decrement = y--; // Decrement (post-decrement)
```

Comparison Operators:

Comparison operators compare two values and return a Boolean result.

```
let a = 10;
let b = 5;

console.log(a > b); // Greater than
console.log(a < b); // Less than
console.log(a >= b); // Greater than or equal to
console.log(a <= b); // Less than or equal to
console.log(a === b); // Equal to (strict equality)
console.log(a !== b); // Not equal to (strict inequality)
```

Logical Operators:

Logical operators perform logical operations and return a Boolean result.

```
let isTrue = true;
let isFalse = false;

console.log(isTrue && isFalse); // Logical AND
console.log(isTrue || isFalse); // Logical OR
console.log(!isTrue); // Logical NOT
```

Assignment Operators:

Assignment operators assign values to variables.

```
let a = 10;
let b = 5;

a += b; // Equivalent to: a = a + b; (addition assignment)
a -= b; // Equivalent to: a = a - b; (subtraction assignment)
```

```
a *= b; // Equivalent to: a = a * b; (multiplication assignment)
a /= b; // Equivalent to: a = a / b; (division assignment)
```

Conditional (Ternary) Operator:

The conditional operator provides a shorthand way of writing an if-else statement.

```
let age = 20;
let canVote = (age >= 18) ? 'Yes' : 'No';
console.log(canVote);
```

String Concatenation:

The + operator is also used for string concatenation.

```
let firstName = 'John';
let lastName = 'Doe';
let fullName = firstName + ' ' + lastName;
console.log(fullName); // Output: John Doe
```

typeof Operator:

The typeof operator returns a string indicating the data type of an operand.

```
let num = 42;
let str = 'Hello';

console.log(typeof num); // Output: number
console.log(typeof str); // Output: string
```

Grouping Operator:

Parentheses () are used for grouping expressions and controlling the order of operations.

```
let result = (10 + 5) * 2; // Output: 30
```

These are some of the basic operators and expressions in JavaScript. By combining these operators in various ways, you can create complex expressions to perform a wide range of tasks in your JavaScript code.

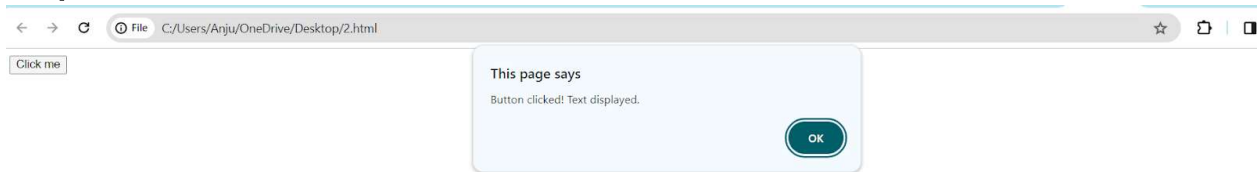
Here is a simple example to click a button and display some text with a simple display function.

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Button Click Example</title>
</head>
```

```
<body>
  <button id="myButton" onclick="displayText()">Click me</button>
  <script>
    // Function to display text
    function displayText() {
      alert('Button clicked! Text displayed.');
```

Output:



We will learn more about functions in the next session.

IX. HTML event attributes

When a browser reacts to user action, then it is called as an event. For example, when you click on the submit button, then if the browser displays an information box.

Following is a table of event attributes, using these attributes you can perform several events.

Form Event Attributes

Form event occurs when the user performs some action within the form such as submitting the form, selecting input field, etc.

The form events can be used with any element, but these are mainly used with HTML form elements.

Following is the list of all Form Event attributes:

Attribute	Description
onblur	Executed the script when form element loses the focus.
onchange	Executed the script when the value of the element is changed.

onfocus	Trigger an event when the element gets focused.
oninput	Executed the script when the user enters input to the element.
oninvalid	Execute the script when the element does not satisfy its predefined constraints.
onreset	Triggers the event when the user reset the form element values.
onsearch	Triggers the event when a search field receives some input.
onselect	Triggers the event when the user has selected some text.
onsubmit	Triggers the event when a form is submitted.

Mouse Event Attributes

Attribute	Description
onclick	Trigger the event when the mouse clicks on the element.
ondblclick	Trigger the event when mouse double-click occurs on the element.
onmousedown	Trigger the event when the mouse button is pressed on the element.
onmousemove	Trigger the event when the mouse pointer moves over the element.
onmouseout	Trigger the event when the mouse moves outside the element.
onmouseover	Trigger the event when the mouse moves onto the element.
onmouseup	Trigger the event when the mouse button is released.
onmousewheel	Deprecated. Use the onwheel attribute.

onwheel	Trigger the event when the mouse wheel rolls up or down on the element
---------	--

Example of mouse event:

```

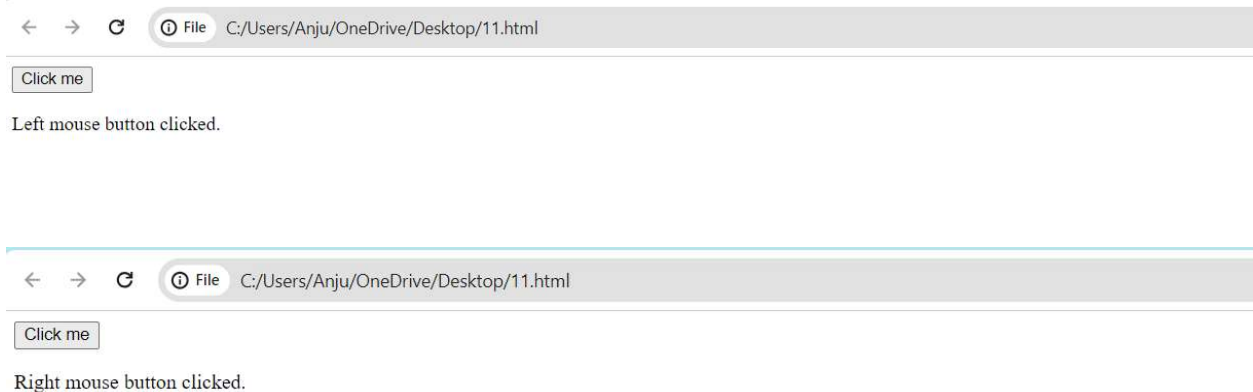
<!DOCTYPE html>
<html>
<head>
  <title>JS Mouse Events - Button Demo</title>
</head>
<body>
  <button id="btn">Click me</button>
  <p id="message"></p>
  <script>
    let btn = document.querySelector('#btn');

    // disable context menu when right-mouse clicked
    btn.addEventListener('contextmenu', (e) => {
      e.preventDefault();
    });

    // show the mouse event message
    btn.addEventListener('mouseup', (e) => {
      let msg = document.querySelector('#message');
      switch (e.button) {
        case 0:
          msg.textContent = 'Left mouse button clicked.';
          break;
        case 1:
          msg.textContent = 'Middle mouse button clicked.';
          break;
        case 2:
          msg.textContent = 'Right mouse button clicked.';
          break;
        default:
          msg.textContent = `Unknown mouse button code: ${event.button}`;
      }
    });
  </script>
</body>
</html>

```

Output:



We will learn more about event driven programming in the upcoming sessions.

ChatGPT Exercise:

Please visit the site - <https://chat.openai.com/> and create your account if not yet created. Next, start with the first question on the prompt:

I am learning HTML , JavaScript and CSS. Could you provide me with an HTML code with JavaScript that creates a simple calculator?

I am learning HTML , JavaScript and CSS. Could you provide me with an HTML code with JavaScript that creates a simple calculator?



Submit the query by clicking the arrow icon. Now go through the response received from the AI engine. Output will be something like the following screenshot. But please note that there can be some minor programming errors in this free version of ChatGPT. Go through the code and correct the code if required.



ChatGPT

Certainly! Here's a simple HTML code with JavaScript to create a basic calculator:

```
html
Copy code

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Simple Calculator</title>

  <style>
    input,
    button {
      width: 50px;
      height: 50px;
      margin: 5px;
      font-size: 18px;
    }

    #result {
      width: 212px;
      margin: 5px;
```

After the 2 hr ILT, the student has to do a 1 hour live lab. Please refer to the Session 6 Lab doc in the LMS.